

Artifact: PartiBench - A Benchmarking and Distributed Inference Framework for CNN Partitioning across the Computing Continuum

Nikolaos Papadakis*, Alexandros Angourakis†, Kostas Magoutis†‡, Georgios Bouloukakis§*

*Télécom SudParis, Institut Polytechnique de Paris, France

†Computer Science Department, University of Crete, Greece

‡Institute of Computer Science (ICS), Foundation for Research and Technology – Hellas (FORTH), Greece

§Department of Electrical and Computer Engineering, University of Patras, Greece

nikolaos.papadakis@telecom-sudparis.eu, alexagou@yahoo.gr, magoutis@ics.forth.gr, gbouloukakis@upatras.gr

Abstract—This artifact accompanies the paper “*Profiling Neural Network Partitioning Strategies for Inference across the Computing Continuum*.” [1] It provides two tools: PartiBench, which profiles VGG16 [2] layer blocks on individual nodes under no-split, input-split, and filter-split configurations; and a Distributed CNN Inference Orchestrator, which deploys and runs a partitioned VGG16 pipeline across IoT-Edge-Cloud nodes. All raw measurements from the paper’s EC2 experiments are included, along with a script that regenerates every paper figure. Full replication requires a multi-node Linux setup; all paper figures can be reproduced on a single machine in under a minute using the provided past run measurements.

I. AVAILABILITY

The artifact is publicly available at:

<https://github.com/satrai-lab/partibench/>

The repository contains all source code, pre-collected benchmark data, original figures, and this document. It is released under the MIT license.

II. SYSTEM DESCRIPTION

PartiBench runs locally on a single node and profiles every VGG16 layer block (blocks 0–15) under four partition factors (25%, 50%, 75%, 100%) and three strategies: no split, input split (data parallelism), and filter split (model parallelism). Each configuration is measured 6 times; the first run is discarded as warm-up. The tool outputs a per-node JSON file containing mean, median, and standard deviation of execution time, memory footprint, and input/output tensor sizes.

The Distributed CNN Inference Orchestrator accepts a `benchmark.json` (merged from all per-node files) and a placement specification (`hs.txt`, one JSON line per VGG16 block). It resolves the strategy into a concrete component layout and deploys three process types on each node via a lightweight daemon:

- **Producer**: sends the input image into the pipeline and records end-to-end latency over 16 runs.
- **Block**: receives an input tensor, runs its assigned VGG16 layers, and forwards the output.

- **Bridge**: collects and merges outputs from upstream Block processes, then forwards to the next stage.

Network delays are simulated at the application level via `net_rules.tsv`, so the full system can be tested on a single or multiple machines without real/actual bandwidth constraints.

III. ARTIFACT CONTENTS AND DATA

TABLE I
REPOSITORY STRUCTURE AND FILE DESCRIPTIONS.

Path	Description
<code>core.py</code>	Sockets, model loading, partitioning math
<code>benchmarking/benchmark.py</code>	PartiBench profiler; writes <code><node>.json</code>
<code>benchmarking/merge_bench.py</code>	Merges per-node JSONs into <code>benchmark.json</code>
<code>orchestrator/place.py</code>	Resolves <code>hs.txt</code> ; configures all nodes
<code>orchestrator/local_deploy.py</code>	Node daemon; spawns Producer/Block-/Bridge
<code>orchestrator/hs.txt</code>	Placement spec (one JSON line per VGG16 block)
<code>config/local_example/</code>	tc bandwidth shaping, cgroup throttle
<code>results/exp_analytical_data.txt</code>	Self-contained 2-node loopback simulation
<code>results/exp_analytical_data.txt</code>	Raw EC2 per-node execution times (ms), memory tables, benchmarking runtimes
<code>results/experiments.txt</code>	Lab notebook: all runs and observations
<code>results/plot_paper_figures.py</code>	Regenerates all five paper figures
<code>results/PLOTS/final/</code>	Experiment figures

Pre-collected data. All measurements that are in `results/exp_analytical_data.txt` were collected on Amazon EC2 (Amazon Linux 2023) using the same PartiBench tool provided in this artifact. Each entry reports the raw per-run timing arrays (15 runs), mean, and standard deviation. Execution times were obtained with `benchmark.py` (5 post-warm-up runs per configuration); the benchmarking runtimes section records wall-clock time for full profiling runs. No external datasets are used: VGG16 weights are downloaded automatically by GluonCV [3] and the single test image (`data/ship.png`) is included in the repository.

IV. SYSTEM REQUIREMENTS

Minimum (figure reproduction & local deployment only). Any Linux or macOS x86-64 machine with Python 3 and

internet access (for VGG16 weight download, ≈ 500 MB, required only for the local simulation). No GPU or special hardware is needed.

Full replication (paper § VI). Multiple Linux x86-64 hosts with TCP connectivity on ports 5000+ and 6999. The paper used Amazon EC2 t3.large (IoT/Edge) and t3.xlarge (Cloud) instances. CPU throttling (cgroup) and bandwidth shaping (tc/iproute2) require root. No GPUs are used.

V. ENVIRONMENT SETUP

Install all dependencies in a virtual environment e.g MXNet 1.9.1 [4] (see requirements.txt).

```
python3 -m venv partibench_env
source partibench_env/bin/activate
pip install -r requirements.txt
```

VI. STEP-BY-STEP GUIDE

A. Reproduce Paper Figures (<1 min)

```
# All five figures -> results/PLOTS/reproduced/
python results/plot_paper_figures.py

# Specific figures only (6=activation sizes, 7=pipelined,
# 8=input growth, 9=input partitioning, 10=filter vs input)
python results/plot_paper_figures.py --figs 6 7 9

# Use your own benchmark data
python results/plot_paper_figures.py --bench path/to/
benchmark.json
```

B. Local 2-Node Simulation (≈ 7 min, single machine)

Run all commands from local_example/:

```
python setup_local.py # Step 1: config
cp net_rules/node_A_rules.tsv net_rules.tsv
python ../benchmarking/benchmark.py node_A vgg16 True # Step
2: profile
cp node_A.json benchmarks/nodes/node_A.json
python ../benchmarking/merge_bench.py # Step 3: merge
```

```
# Step 4: start node agents (two separate terminals)
python ../orchestrator/local_deploy.py --ip 127.0.0.2 #
terminal 1
python ../orchestrator/local_deploy.py --ip 127.0.0.1 #
terminal 2

# Step 5: deploy strategy (third terminal)
python ../orchestrator/place.py --start-node node_A \
--graph cc_graph.gml \
--bench benchmarks/benchmark.json --hs hs.txt
```

In node_A's terminal, type `s` to trigger inference.

C. Full Multi-Node Evaluation (paper § VI)

On each node: copy `net_rules/<node>_rules.tsv` as `net_rules.tsv`, run `sudo config/configure.sh 33 (IoT) or sudo config/configure.sh (Edge/Cloud)`, then run `benchmark.py`. Collect all `<node>.json` files, merge, and orchestrate as above. The paper's EC2 node configuration is given in Table I of the main paper.

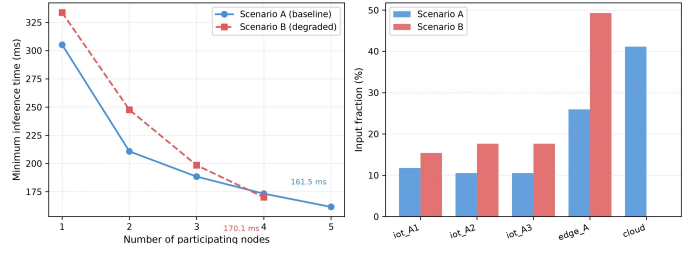


Fig. 1. Theoretical inference time (left) and optimal input distribution per node (right) for input-partitioned VGG16 blocks 0–6. Scenario A: baseline. Scenario B: +80 ms edge–cloud latency, +200 ms IoT delay, merger moved to edge_A.

VII. EXPECTED OUTPUT AND ASSESSMENT

Figure reproduction. Running `plot_paper_figures.py` produces five PDFs in `results/PLOTS/reproduced/`. Figs. 2 and 1 show two representative outputs. Figures derived from `benchmark.json` will match the paper when using the bundled pre-collected data (there might be some small aesthetic differences). Fig. 1 shows Fig. 9 from the paper as reproduced by this script: the theoretical minimum inference time for input-partitioned blocks 0–6 as nodes are added (Scenario A: 161.5 ms with five nodes vs. 305.1 ms cloud-only; Scenario B: 170.1 ms under degraded WAN), with the optimal input fraction per node shown as bar plots.

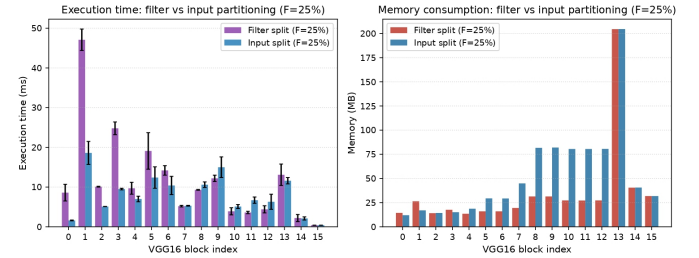


Fig. 2. Memory consumption and average execution time of VGG16 block partitions with partition factor $F = 25$

Local simulation. After triggering inference, the expected output is:

```
PRODUCER: Inference took: XXXX ms
PRODUCER: Results:
container ship, container vessel: 84.32%
wreck: 5.17% | lifeboat: 3.91%
```

The top-3 predictions are deterministic. End-to-end time will vary by machine; on a modern laptop, expect 200–800 ms for the 2-node loopback configuration. Times reported in the paper were measured on EC2 with `tc` bandwidth shaping applied; loopback results are not directly comparable.

Full replication. Results should match the paper within one standard deviation (reported in `exp_analytical_data.txt`). Variance between runs is expected; each timing is averaged over 5 post-warm-up runs.

REFERENCES

- [1] N. Papadakis, A. Angourakis, K. Magoutis, and G. Bouloukakakis, "Profiling neural network partitioning strategies for inference across the computing continuum," 2026.
- [2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [3] J. Z. Zhang *et al.*, *GluonCV: A Deep Learning Toolkit for Computer Vision*, 2021, version 0.10.0. [Online]. Available: <https://cv.gluon.ai/>
- [4] T. Chen, M. Li, Y. Li, M. Lin, N. Wang, M. Wang, T. Xiao, B. Xu, C. Zhang, and Z. Zhang, "Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems," 2015. [Online]. Available: <https://arxiv.org/abs/1512.01274>